

Physics-Informed Neural Networks

Business Applications & Industry Value

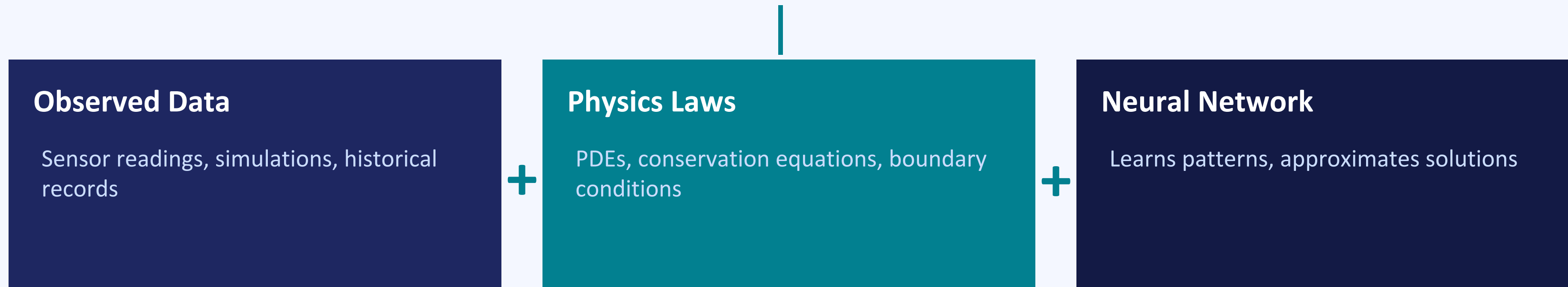
Where AI meets the laws of physics to deliver measurable ROI



What is a PINN?

Traditional simulation: accurate but slow & expensive

Pure AI/ML: fast but physically wrong — unreliable for engineering decisions



= PINN: Simulation-speed predictions that obey the laws of physics

Works with sparse data · No mesh required · Physically valid outputs · 100× faster than classical solvers

How PINNs Work — Heat Equation Example

Consider the one-dimensional heat equation: $\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$

$u(x, t)$ = temperature

(α) = thermal diffusivity constant

Step 1: Neural Net Prediction

NN takes (x, t) as input and outputs predicted temperature $\hat{u}(x, t)$.

$$NN(x, t) = \hat{u}(x, t)$$

Step 2: Auto-Differentiation

Derivatives $\partial \hat{u} / \partial t$ and $\partial^2 \hat{u} / \partial x^2$ are computed automatically via backprop.

Step 3: PDE Residual

Residual $f(x, t) = \partial \hat{u} / \partial t - \alpha \partial^2 \hat{u} / \partial x^2$ measures physics violation.

$$f(x, t) = \frac{\partial \hat{u}}{\partial t} - \alpha \frac{\partial^2 \hat{u}}{\partial x^2}$$

Step 4: Loss Minimization

Optimizer minimizes combined loss until PDE residual $\rightarrow 0$.

PINN Loss Function

$$\mathcal{L} = \mathcal{L}_{data} + \mathcal{L}_{physics} + \mathcal{L}_{boundary}$$

Data Loss

- Fits known observations
- Minimizes prediction error
- Standard MSE form

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N (\hat{u}_i - u_i)^2$$

u_i = true temperature

$\hat{u}_i$ = predicted temperature

Physics Loss

- Enforces PDE satisfaction
- Sampled at collocation pts
- Penalizes residual violation

$$\mathcal{L}_{physics} = \frac{1}{N_f} \sum_{i=1}^{N_f} f(x_i, t_i)^2$$

(N_f) = number of collocation points

$f(x_i, t_i)$ = PDE residual at sampled points

Boundary Loss

- Enforces boundary conditions
- Ensures valid solutions
- E.g., $u(0,t) = 0$

$$\mathcal{L}_{boundary} = \frac{1}{N_b} \sum_{i=1}^{N_b} (\hat{u}_b - u_b)^2$$

$\hat{u}_b$ = predicted boundary value

u_b = actual boundary value

Advantages of PINNs

- Incorporates physics directly into learning
- Requires less labeled data
- Mesh-free approach
- Continuous solution representation
- Useful for inverse problems
- Faster inference after training
- Reduces computational and prototype cost
- Enables real-time predictions

Industry Applications

Aerospace

- Aerodynamic optimization
- Airflow prediction
- Turbulence modeling
- Reduced-order modeling

Automotive

- Battery thermal analysis
- Vehicle aerodynamics
- Heat transfer simulation
- Fluid flow optimization

Healthcare

- Blood flow simulation
- Biomedical fluid dynamics
- Cardiovascular modeling
- Drug diffusion analysis

Climate

- Weather prediction
- Ocean simulation
- Climate forecasting
- Atmospheric modeling

Digital Twins

- Real-time monitoring
- Predictive maintenance
- Industrial simulations

Energy

- Reservoir modeling
- Pipeline flow analysis
- Wind farm optimization
- Heat transfer systems

Current Industrial Adoption

NVIDIA

Modulus / PhysicsNeMo

GPU-accelerated framework for PINNs, neural operators, and multi-physics simulations. Powers fluid, climate, and digital twin workflows.

Siemens

Digital Twins & Smart Factories

PINN-based industrial monitoring and predictive maintenance. Continuously simulates real systems using live sensor data.

Boeing / NASA / Airbus

Aerospace CFD

Replacing costly CFD runs (hours → seconds) for airflow, turbulence, and aerodynamic design iteration.

EV & Auto OEMs

Battery & Thermal

Fast surrogate models for battery cooling, airflow, and thermal prediction — reducing prototype cost.

Evolution of PINNs

A Gradual Revolution

PINNs did not emerge suddenly. Researchers first used neural networks purely for pattern recognition before exploring whether they could solve differential equations directly.

1990s

Early works used NNs as function approximators to minimise PDE residuals and enforce boundary conditions — but small networks, no GPU, and no autodiff made scaling impossible.

2000s

Methods stalled. Without automatic differentiation, computing PDE derivatives through a network was impractical. TensorFlow and PyTorch hadn't arrived yet.

The 2017 Breakthrough

TensorFlow & PyTorch introduced autodiff and GPU compute. Raissi et al. unified everything into one framework:

- 1 Deep learning
- 2 PDE residual minimisation
- 3 Boundary condition enforcement
- 4 Automatic differentiation

The field is born.

Rapid Expansion

2019

DeepXDE

First major open-source PINN library — made the framework accessible to all researchers.

2020

Fourier Neural Operators

Learned entire solution operators, not just individual solutions — massive scalability leap.

2021

NVIDIA Modulus

Industrial GPU platform brought PINNs into real engineering workflows.

2022+

Adaptive & Multi-scale

Tackled convergence, turbulence, and complex PDEs via hybrid architectures.

Evolution Table

Year	Paper / Work	Main Contribution	Key Innovation Introduced
1994	Dissanayake & Phan-Thien	First major attempt to use neural networks for solving PDEs	Introduced neural networks as PDE approximators with physics-based loss
1998	Lagaris et al.	Improved neural PDE solving with exact boundary handling	Introduced trial solution functions that automatically satisfy boundary conditions
2000	Lagaris et al.	Extended neural PDE methods to complex geometries	Enabled handling of irregular boundaries and more realistic domains
2017	Raissi et al. — Physics-Informed Neural Networks (PINNs)	Formal introduction of modern PINNs	Combined deep learning, automatic differentiation, PDE residual loss, and physics constraints
2019	DeepXDE	First major PINN framework/library	Simplified implementation of PINNs, PDEs, geometries, and boundary conditions
2020	Fourier Neural Operators (FNO)	Introduced neural operator learning for PDEs	Learned entire PDE operators instead of single solutions, improving scalability
2020	DeepONet	General operator learning framework	Learned mappings between function spaces for solving families of PDEs
2021	NVIDIA Modulus / PhysicsNeMo	Industrial scientific AI framework	Large-scale GPU-accelerated PINNs and scientific ML for real-world engineering applications
2022+	Adaptive & Multi-Scale PINNs	Improved training and scalability of PINNs	Adaptive sampling, better convergence, and handling of complex multi-scale physics
2024+	Scientific Foundation Models	Large pretrained AI models for scientific computing	Generalized physics learning for digital twins, climate modeling, and real-time simulation

The Business Case

Why PINNs deliver ROI that pure ML cannot

100×

Faster than classical
numerical solvers

Hours → seconds for simulation

82%

Less compute cost
in climate modeling

vs. CMIP6 benchmark models

37%

Lower prediction
error in key domains

Ocean temperature anomalies

<1%

Reservoir prediction
error with sparse data

vs. 3–5% for pure ML

When to choose PINNs over standard ML:

- You have limited labeled data but know the governing physics
- Predictions MUST be physically valid — no impossible outputs
- Classical simulation is too slow for real-time or design optimization
- You need to solve inverse problems — infer hidden system parameters from measurements